

1- What is Playwright and how does it differ from Selenium and Cypress?

Playwright is an open-source end-to-end automation framework developed by Microsoft. It supports all modern browsers—Chromium, Firefox, and WebKit—and allows us to write tests in JavaScript, TypeScript, Python, C#, and Java.

Compared to Selenium, Playwright is faster and more reliable because it communicates directly with browser DevTools protocol instead of relying on WebDriver. It also has smart features like auto-waiting and built-in parallel execution, which reduce flakiness.

If I compare it with Cypress, the main advantage is cross-browser and multi-tab support. Cypress has limitations like single-browser type execution and difficulty handling multiple domains or iframes. Playwright overcomes these limitations and also provides capabilities like network interception, tracing, and API testing.

So in short, Playwright is a modern, faster, and more feature-rich alternative to Selenium and Cypress, and it's particularly good for scalable enterprise automation.

2. Explain the Playwright architecture?

Playwright architecture is built around three main layers:

Client Library (Test Runner / Playwright API)

- This is what we use in our code. It provides the test runner, assertions, fixtures, and APIs like page, browser, and context.

Driver / Communication Layer

- The Playwright client talks to the browser using a WebSocket connection. Unlike Selenium, which uses WebDriver protocol, Playwright connects directly to the browser's DevTools protocol. This makes it faster and less flaky.

Browser Layer

- Playwright controls real browsers: Chromium (Chrome/Edge), Firefox, and WebKit (Safari). It also supports multiple browser contexts inside the same browser instance, which is very useful for parallel execution or simulating different users.

On top of that, Playwright also has test runner features like parallel execution, auto-waiting, fixtures, tracing, screenshots, and video recording, all integrated within the same architecture.

So in simple words, the Playwright architecture is a combination of:

A client library (where we write tests),

A driver that communicates directly with browser protocols, and

Real browsers running the tests with high reliability.

3. What are the different locators available in Playwright?

In Playwright, locators are very powerful because they can uniquely identify elements in different ways. Some of the commonly used locators are:

1. CSS Selectors – We can use standard CSS syntax like `.classname`, `#id`, `input[name="email"]`.
2. Text Selector – Playwright allows locating elements directly by visible text, for example: `page.getByText("Login")`.
3. Role-based Selectors – This is based on ARIA roles and is very useful for accessibility, e.g. `page.getByRole('button', { name: 'Submit' })`.
4. Placeholder Selectors – For input fields with placeholder text, e.g. `page.getByPlaceholder("Enter email")`.
5. Label Selectors – To find input elements associated with labels, e.g. `page.getByLabel("Password")`.
6. Alt Text Selectors – To locate images or elements by their alt attribute, e.g. `page.getByAltText("Company Logo")`.
7. Test ID Selectors – Using custom data-testid attributes for stable test locators, e.g. `page.getByTestId("username-field")`.

8. XPath Selectors – Although not recommended as first choice, Playwright supports XPath too, e.g. `page.locator("//div[@id='main']").`

The advantage in Playwright is that it provides semantic locators like `getByRole`, `getByLabel`, `getByPlaceholder`, which make tests more readable, stable, and aligned with accessibility standards.

4. How does `page.locator()` differ from `page.$()` or `page.querySelector()`?

In Playwright, `page.locator()` is the recommended way to work with elements, while `page.$()` or `page.querySelector()` are older style methods.

The key differences are:

Locator Object vs Element Handle

- `page.locator()` returns a Locator object. It's like a smart wrapper that can perform actions (click, fill, type) directly and automatically retries until the element is ready.
- `page.$()` or `page.querySelector()` return an ElementHandle. This is a direct reference to a DOM element at that point in time. It doesn't auto-retry and can become stale if the DOM changes.

Auto-Waiting

- `locator.click()` automatically waits for the element to be visible, enabled, and stable before clicking.
- With `elementHandle.click()`, you need to manually wait or add extra checks, otherwise tests can fail with "element not found" or "element not visible".

Chaining and Reusability

- Locators are chainable. For example:

```
const row = page.locator('table >> tr').nth(2);
await row.locator('td').first().click();
```

This is very useful for dynamic elements.

- ElementHandles don't support this chaining as cleanly.

Best Practice

- Playwright team recommends using `locator()` for all new code because it makes tests more stable, less flaky, and easier to maintain.
- `page.$()` or `page.querySelector()` are still there mainly for compatibility or one-time DOM inspection, but not for regular test actions.

5. How do you handle multiple browser contexts in Playwright?

In Playwright, a browser context is like a fresh, isolated browser profile. Each context has its own cookies, cache, and storage, so it behaves like a separate user session.

To handle multiple contexts, I usually launch a browser once and then create multiple contexts from it. For example:

```
const browser = await playwright.chromium.launch();
const context1 = await browser.newContext();
const context2 = await browser.newContext();
```

```
const page1 = await context1.newPage();
const page2 = await context2.newPage();
```

```
await page1.goto('https://app.example.com');
await page2.goto('https://app.example.com');
```

Here, `page1` and `page2` are completely independent. This is useful when I want to:

- Test multi-user scenarios (like Buyer and Seller login at the same time).
- Run parallel tests faster without launching a separate browser for each test.
- Simulate different environments or permissions in the same test run.

Compared to Selenium, where we typically open separate browsers, Playwright contexts are more lightweight and faster.

So, whenever I need isolation without the overhead of launching new browsers, I go for multiple contexts.

6. What is the difference between `page.click()` and `locator.click()`?

The difference is mainly about what object we are interacting with and how Playwright handles waiting:

`page.click()`

- This is a shortcut method on the whole page object.
- You pass a selector string directly, for example:

```
await page.click('#login');
```

1. – It will find the element, wait for it to be ready, and then click. But if the element changes in the DOM or appears later, sometimes it may be less reliable.

`locator.click()`

- `locator()` returns a Locator object, which is like a smart pointer to the element.
- It supports auto-retries and always re-resolves the element before action. For example:

```
const loginBtn = page.locator('#login');  
await loginBtn.click();
```

2. – This makes it more stable, especially when elements are dynamic or re-render frequently.
3. Best Practice
 - The Playwright team recommends using `locator.click()` instead of `page.click()` because:
 - Locators are reusable and chainable.
 - They auto-wait for visibility, stability, and readiness.
 - They reduce flakiness compared to using raw selectors with `page.click()`.

So, in short: *`page.click()` is a one-time action on a selector string, while `locator.click()` is smarter, reusable, and more stable. I prefer `locator.click()` in real-world frameworks.*

7. How do you launch tests in headed vs headless mode?

By default, Playwright runs tests in headless mode, meaning the browser UI is not visible. If we want to run in headed mode (with the browser UI visible), we just need to change the launch configuration.

For example:

```
// Headless (default)
const browser = await playwright.chromium.launch({ headless: true });
```

```
// Headed
const browser = await playwright.chromium.launch({ headless: false });
```

In the Playwright Test Runner, we can also control it through the config file:

```
// playwright.config.js
use: {
  headless: false, // set to true for headless
}
```

Or even override it via CLI:

```
npx playwright test --headed # runs in headed mode
npx playwright test --headless # runs in headless mode
```

I usually run tests in headed mode during debugging to see the browser actions, and in headless mode in CI/CD pipelines for faster execution.

So, the difference is mainly visibility vs performance — headed for debugging, headless for speed and automation."

8. What is the role of `playwright.config.ts/js`?

The `playwright.config.ts` or `playwright.config.js` file is the central configuration file for Playwright tests. Instead of repeating settings in every test, we define them once here and Playwright applies them globally.

Some important roles of this file are:

1. Test Execution Settings
 - Define global timeout, retries, reporter, parallel execution, etc.
2. Browser & Device Configuration
 - Choose which browser to run (chromium, firefox, webkit), set viewport size, headless vs headed, device emulation, and video/screenshot options.
3. Environment Handling
 - Set baseURL, environment variables, or custom fixtures. For example, I can define baseURL as `https://qa.myapp.com` and just use `page.goto('/')` in my tests.
4. Projects & Parallelism
 - Configure multiple projects to run tests across browsers or devices in parallel. For example, same test suite can run on Chrome, Firefox, and WebKit with one command.
5. Reporters & Trace
 - Enable HTML reports, Allure reports, or Playwright's built-in trace viewer.

For example, a simple config might look like:

```
// playwright.config.js
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    headless: true,
    baseURL: 'https://myapp.com',
    screenshot: 'on',
    video: 'retain-on-failure',
  },
  retries: 2,
  reporter: [['html', { open: 'never' }]],
});
```

So, in short: the config file is like the control center of Playwright tests — it ensures consistency, reduces duplication, and makes it easy to manage test execution across environments and browsers.

9. How do you handle waits in Playwright?

In Playwright, the good thing is that we usually don't need to add explicit waits because Playwright has auto-waiting built in. That means before any action like click, fill, or type, it automatically waits for the element to be attached to DOM, visible, stable, and enabled.

But apart from auto-waiting, there are different ways to handle waits when needed:

Locator-based Waits (Recommended)

- Use Playwright's expect API with conditions:

```
await expect(page.locator('#login')).toBeVisible();
```

- Or wait for specific state:

```
await page.locator('#login').waitFor({ state: 'visible' });
```

1.

Page Waits

- For navigation:

```
await page.waitForURL('**/dashboard');  
await page.waitForLoadState('networkidle');
```

2.

Wait for Specific Events

- Like waiting for a response or request:

```
await page.waitForResponse(res => res.url().includes('/api/login') && res.status() === 200);
```

3.

Manual Waits (Not recommended, only for debugging)

```
await page.waitForTimeout(3000); // pause for 3s
```

4.

Best Practice: I avoid hard-coded waits and prefer auto-waits, assertions, or event-based waits. This keeps the tests fast and stable, and avoids flakiness.

10. Explain the difference between `expect(locator).toBeVisible()` and `await locator.isVisible()`.

The main difference is that one is an assertion and the other is a boolean check.

`expect(locator).toBeVisible()`

- This is part of Playwright's test assertions.
- It will auto-wait until the element becomes visible or the timeout is reached.
- If the element does not become visible within the timeout, the test will fail with a proper error message.
- Example:

```
await expect(page.locator('#loginBtn')).toBeVisible();
```

1. This will keep retrying until the login button is visible.

`await locator.isVisible()`

- This is a method on the Locator API.
- It returns a boolean value (true if visible, false if not).
- It does not throw an error by itself; we need to handle the result.
- Example:

```
if (await page.locator('#loginBtn').isVisible()) {  
  await page.click('#loginBtn');  
}
```

When to use what:

- If I want to assert and fail the test when the element isn't visible → use `expect(locator).toBeVisible()`.
- If I just want to check conditionally (for example, "if visible, then click") → use `await locator.isVisible()`.

So, in short:

`expect(...).toBeVisible()` = assertion with auto-wait + failure on mismatch

`locator.isVisible()` = boolean check for conditional logic.

Intermediate Playwright

11. How do you handle multiple tabs or windows in Playwright?

In Playwright, a new tab or window is represented as a new Page object. By default, Playwright doesn't automatically switch context like Selenium; instead, we need to explicitly listen for the new page and then work with it.

For example, if clicking a link opens a new tab:

```
// Listen for new page (tab/window)
const [newPage] = await Promise.all([
  context.waitForEvent('page'),
  page.click('a[target="_blank"]'), // action that opens the new tab
]);
```

```
// Now work with the new tab
await newPage.waitForLoadState();
await newPage.goto('https://example.com');
```

Key points:

- Each tab is just another Page object inside the same Browser Context.
- We use `context.waitForEvent('page')` to capture it reliably.
- We can keep multiple Page objects and interact with them independently.

Use cases:

- Testing flows like payment gateways (which open new windows).
- Handling social login (Google/Facebook pop-ups).
- Validating links that open in new tabs.

So, unlike Selenium where we switch window handles, in Playwright we simply capture the new Page object and use it directly.

12. Explain how you deal with file upload and file download in Playwright.

Playwright has built-in support for both file upload and file download.

1. File Upload

- We don't interact with the system file dialog. Instead, we set the file directly on the input element.

- Example:

```
// Upload a single file
await page.setInputFiles('input[type="file"]', 'tests/data/resume.pdf');
```

```
// Upload multiple files
await page.setInputFiles('input[type="file"]', [
  'tests/data/file1.pdf',
  'tests/data/file2.png'
]);
```

- This is reliable because Playwright bypasses the OS dialog and directly simulates file selection.

2. File Download

- For downloads, Playwright provides a `page.waitForEvent('download')` method.

- Example:

```
const [download] = await Promise.all([
  page.waitForEvent('download'), // wait for download to start
  page.click('text=Download Report') // action that triggers download
]);
```

```
// Save the downloaded file
const filePath = await download.path();
```

```
await download.saveAs('downloads/report.pdf');
```

– We can also get file name, suggested path, or save it in a custom location.

Real-world usage:

- Upload → resumes, profile pictures, or bulk import files.
- Download → validating invoices, reports, or exported data.

So in short:

File upload is done with `setInputFiles()`

File download is handled via `page.waitForEvent('download')` and `saveAs()`.

13. What is the use of fixtures in Playwright?

In Playwright, fixtures are a way to set up and share reusable test context. They help manage resources like browser, page, test data, or custom objects so we don't have to repeat setup code in every test.

For example, Playwright provides built-in fixtures like page, browser, and context. So in a test I can just write:

```
import { test, expect } from '@playwright/test';

test('login test', async ({ page }) => {
  await page.goto('https://app.example.com');
  await page.fill('#username', 'admin');
  await page.fill('#password', 'password');
  await page.click('button[type="submit"]');
  await expect(page).toHaveURL('/dashboard');
});
```

Here, I didn't create the browser or page myself — they came from the fixtures.

We can also create custom fixtures. For example, a fixture for `adminPage` that logs in before each test:

```
export const test = base.extend({
  adminPage: async ({ page }, use) => {
    await page.goto('/login');
    await page.fill('#username', 'admin');
    await page.fill('#password', 'password');
    await page.click('button[type="submit"]');
    await use(page); // provide logged-in page to tests
  },
});
```

Now, when I want to use this adminPage fixture in a test, I can simply inject it:

```
import { test, expect } from './fixtures/admin.fixture';

test('verify admin dashboard', async ({ adminPage }) => {
  await expect(adminPage).toHaveURL('/dashboard');
  await expect(adminPage.locator('h1')).toHaveText('Admin Dashboard');
});
```

Here the test doesn't need to handle login steps — it just receives a logged-in adminPage.

Use cases of fixtures:

- Managing browser/page lifecycle.
- Injecting test data or API clients.
- Creating logged-in sessions.
- Running setup/teardown logic in a structured way.

So, in short: fixtures in Playwright are like building blocks for test setup and dependencies. They make tests more reusable, maintainable, and less flaky.

14. How do you run Playwright tests in parallel?

Playwright has built-in support for running tests in parallel, and we can achieve this in two main ways:

1. Parallel Execution by Default

- Each test file runs in parallel with others, because Playwright spawns workers (separate processes).
- We can control the number of workers in the config file:

```
// playwright.config.js
export default {
  workers: 4, // run with 4 parallel workers
};
```

Or from CLI:

```
npx playwright test --workers=4
```

2. Projects for Cross-Browser Parallelism

- We can define multiple projects (e.g., Chrome, Firefox, WebKit), and Playwright will run them in parallel:

```
export default {
  projects: [
    { name: 'chromium', use: { browserName: 'chromium' } },
    { name: 'firefox', use: { browserName: 'firefox' } },
    { name: 'webkit', use: { browserName: 'webkit' } },
  ],
};
```

Running this will execute the same test suite across all browsers at the same time.

3. Test-level Parallelism

- Inside a test file, tests can also run concurrently if they don't share state.
- Example:

```
import { test } from '@playwright/test';

test.describe.configure({ mode: 'parallel' });

test('test A', async ({ page }) => { /* ... */ });
test('test B', async ({ page }) => { /* ... */ });
```

This allows tests in the same file to run in parallel instead of sequentially.

Best Practices:

- Keep tests independent (no shared state).
- Use isolated fixtures or contexts so tests don't interfere with each other.
- In CI/CD, tune workers based on machine cores to optimize execution time.

So in short: Playwright runs tests in parallel by default at the file level, and we can scale it further using workers, projects, and parallel test suites."

15. How do you configure retries for failed tests?

In Playwright, we can configure retries so that a failed test is re-run automatically before being marked as failed. This is very useful for handling flaky tests.

There are a few ways to configure retries:

1. In the Config File (Global Setting)

- We can define retries for all tests in playwright.config.js/ts:

```
// playwright.config.js
export default {
  retries: 2, // retry failed tests up to 2 times
};
```

2. From CLI

- We can override retries when running tests:

```
npx playwright test --retries=3
```

3. Per-test Basis

- We can also configure retries inside the test itself:

```
import { test } from '@playwright/test';

test('flaky test example', async ({ page }) => {
  // test logic
}).retry(2); // this test will retry 2 times if it fails
```

Best Practices:

- Use retries only for known flaky scenarios (like network delays or animations).
- Fix the root cause instead of relying too much on retries.
- Combine retries with trace and video recording so failures are easier to debug.

So, in short: retries can be set at global, CLI, or per-test level, and they help reduce flaky failures, especially in CI/CD pipelines."

16. How do you use custom test fixtures?

In Playwright, fixtures allow us to share setup and teardown logic across tests. Apart from built-in fixtures like page or context, we can also create custom fixtures to make tests cleaner and more reusable.

1. Creating a Custom Fixture

For example, if I want a fixture that provides a logged-in admin user:

```
import { test as base } from '@playwright/test';

export const test = base.extend({
  adminPage: async ({ page }, use) => {
    // login steps
    await page.goto('/login');
    await page.fill('#username', 'admin');
    await page.fill('#password', 'password123');
    await page.click('button[type="submit"]');
```

```
// pass the logged-in page to the test
await use(page);
},
});
```

2. Using the Custom Fixture in a Test

Now I can inject adminPage directly in my test without repeating login code:

```
import { test, expect } from './fixtures/admin.fixture';

test('verify admin dashboard', async ({ adminPage }) => {
  await expect(adminPage).toHaveURL('/dashboard');
  await expect(adminPage.locator('h1')).toHaveText('Admin Dashboard');
});
```

3. Benefits / Use Cases

- Avoids repeating boilerplate code like logins.
- Helps inject test data, API clients, or DB connections.
- Makes tests more readable, modular, and maintainable.
- Works well with Playwright's auto-fixture lifecycle (setup before test, teardown after).

So in short: custom fixtures in Playwright are like reusable building blocks. Instead of repeating setup steps in every test, we define them once and inject them where needed.

17. How do you record and generate code using Playwright Codegen?

Playwright provides a built-in tool called Codegen that allows us to record browser actions and automatically generate test scripts. It's very useful for getting started quickly or for learning selectors.

1. Running Codegen from CLI

I can start code generation with:

```
npx playwright codegen https://example.com
```

This will:

- Launch a browser,
- Record all actions (clicks, typing, navigation),
- Generate Playwright code in real-time.

2. Generating Code in Different Languages

By default, it generates TypeScript/JavaScript, but I can specify:

```
npx playwright codegen --target=python https://example.com
```

```
npx playwright codegen --target=java https://example.com
```

```
npx playwright codegen --target=csharp https://example.com
```

3. Saving the Code

Once recording is done, I can copy the code or save it into a test file.

4. Real-world Usage

- Quick prototype of a new test.
- Helps non-technical QA create initial scripts.
- Identifies stable locators by showing getByRole, getByText, etc.

Best Practice: I usually don't rely on Codegen for production-level scripts, but I use it to bootstrap tests, grab selectors, or train new team members. After recording, I refactor the code into the framework with fixtures, POM, and reusable utilities.

So, in short: Codegen is a handy recorder that accelerates writing Playwright scripts, but the recorded code should be treated as a starting point, not the final framework."

18. What are trace viewer and video recording in Playwright?

Playwright provides trace viewer and video recording features to help us debug and analyze test runs, especially when tests fail.

1. Trace Viewer

- A trace is like a detailed log of the test execution.
- It captures screenshots, DOM snapshots, network requests, console logs, and steps during the test.
- To enable it, I configure in `playwright.config.js`:

```
use: {  
  trace: 'on-first-retry', // options: 'on', 'on-first-retry', 'retain-on-failure'  
}
```

- After the run, I can open the trace with:

```
npx playwright show-trace trace.zip
```

- It gives a timeline view where I can replay the test step-by-step and see what happened at each point.

2. Video Recording

- Playwright can record a video of the test execution for debugging.

- In config:

```
use: {  
  video: 'retain-on-failure', // options: 'on', 'off', 'retain-on-failure'  
}
```

- The recorded video helps when debugging in CI/CD pipelines where we cannot see the actual run.

3. Real-world Usage

- Trace viewer → best for step-by-step debugging, analyzing failures, and sharing artifacts with the team.
- Video recording → useful for visual verification, reproducing issues, or reporting bugs to developers.

So in short:

Trace Viewer = detailed test execution timeline with snapshots & logs.

Video Recording = actual playback of the test run.

Both are powerful debugging tools that make tests more maintainable and failures easier to investigate."

19. How do you debug a Playwright test effectively?

I follow a layered approach to debug Playwright tests.

1) Run visibly & slower

Headed + slowMo to watch what's happening:

```
npx playwright test --headed
```

```
// playwright.config.js
```

```
use: { headless: false, launchOptions: { slowMo: 200 } }
```

2) Add a breakpoint

add a breakpoint in code:

```
await page.pause(); // opens inspector at this line
```

3) Narrow the scope

Run just the failing test/file:

```
npx playwright test tests/login.spec.ts:25
```

Or mark a single test:

```
test.only('repro', async ({ page }) => { /* ... */ });
```

4) Strengthen signals with assertions

Prefer expect assertions that auto-wait (reduce flakiness) and add context:

```
await expect(page.getByRole('button', { name: 'Login' })).toBeVisible();
```

5) Collect rich artefacts (on failure or retry)

Trace for step-by-step replay; Video/Screenshots for visuals:

```
// playwright.config.js
use: {
  trace: 'on-first-retry',
  video: 'retain-on-failure',
  screenshot: 'only-on-failure',
}
```

After run:

```
npx playwright show-trace trace.zip
```

6) Tune timeouts—surgically

Don't blanket-increase global timeouts; adjust locally when needed:

```
test.setTimeout(30000);  
await expect(locator).toBeVisible({ timeout: 5000 });
```

7) Validate selectors

- Prefer `getByRole` / `getByLabel` / `getByTestId` over brittle XPath.

8) Use DevTools when needed

For Chromium, open DevTools with the test:

```
use: { launchOptions: { devtools: true } }
```

With this flow—headed + inspector → scoped repro → assertions/artefacts → logs/network → precise timeouts—I can usually isolate the root cause quickly and make the fix stick.

20. What is the default timeout for expect and elements?

In Playwright, there are two main types of timeouts to know about: element actions/locators and expect assertions.

1. Default Timeout for Elements (Locators & Actions)

- By default, Playwright waits up to 30 seconds (30,000 ms) for actions like click, fill, type, or for an element to appear in the DOM.

Example:

```
await page.click('#login'); // will wait max 30s for element to be ready
```

How to Change:

Change at test level:

```
await page.setDefaultTimeout(10000); // 10s for all element actions
```

Change globally in playwright.config.js:

```
use: { actionTimeout: 10000 } // 10s for each action
```

2. Default Timeout for expect Assertions

- Assertions like `expect(locator).toBeVisible()` also wait by default 5 seconds (5,000 ms) before failing.

How to Change:

Override in a single assertion:

```
await expect(page.locator('#welcome')).toBeVisible({ timeout: 10000 });
```

Change globally in config:

```
expect: {  
  timeout: 10000, // all expect() assertions wait up to 10s  
}
```

3. Test-level Timeout

Apart from action/expect timeouts, we can also set an overall test timeout (default is 30s per test):

```
test.setTimeout(60000); // 60s max for the whole test
```

Summary:

Element actions → 30s by default, configurable with `setDefaultTimeout` or `actionTimeout`.

Expect assertions → 5s by default, configurable per assertion or via `expect.timeout`.

Whole test → 30s by default, configurable with `test.setTimeout()`.

This flexibility helps fine-tune tests without unnecessarily increasing global timeouts."

Framework Design

21. How would you design a Playwright test automation framework for scalability?

I have design a Playwright framework using Page Object Model.

1) Framework structure (clear boundaries)

```
/tests          // specs only (no logic)
/pages          // POM / models
/fixtures       // custom fixtures (auth, data, api clients)
/config         // config file
/Utils         // helpers (reading data from json,excel,csv)
/report        // generated artifacts (html, allure, trace, video)
/playwright.config.ts
```

22. Which design patterns can be applied in a Playwright framework?

We have used one of the design patterns called Page Object Model.

What & Why

- I model each page/feature as a class that encapsulates locators + user-intent actions (e.g., loginAs, addToCart), keeping tests readable, stable, and maintainable.
- Changes in UI selectors are localized to the page object; tests don't break everywhere.

Principles I follow

- Single responsibility: one page/feature per class.
- Expose intent, not mechanics: methods like loginAs(user) instead of exposing raw locators.
- Stable locators: prefer getByRole, getByLabel, or data-testid over brittle XPath.
- Auto-waits inside actions: page methods perform the necessary waits; tests stay clean.
- Light on assertions: put verifications in tests; allow lightweight inline checks only when they belong to the action's contract (e.g., wait for dashboard after loginAs).

```
// pages/LoginPage.js
```

```
import { expect } from '@playwright/test';
```

```
export class LoginPage {
```

```
  constructor(page) {
```

```
    this.page = page;
```

```
    this.email = page.getByLabel('Email');
```

```
    this.password = page.getByLabel('Password');
```

```
    this.submit = page.getByRole('button', { name: 'Sign in' });
```

```
}
```

```
async goto() {  
  await this.page.goto('/login');  
}
```

```
async loginAs(email, pass) {  
  await this.email.fill(email);  
  await this.password.fill(pass);  
  await this.submit.click();  
  // Optional contract check for stability:  
  await expect(this.page).toHaveURL(/dashboard/);  
}  
}
```

```
// tests/login.spec.js
```

```
import { test, expect } from '@playwright/test';
```

```
import { LoginPage } from '../pages/LoginPage.js';
```

```
test('user can log in', async ({ page }) => {  
  const login = new LoginPage(page);  
  await login.goto();  
  await login.loginAs(process.env.USER_EMAIL, process.env.USER_PASS);  
  await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();  
}
```

```
});
```

23. How do you manage test data (JSON, CSV, Excel) in Playwright?

We manage test data in three layers: storage → loader utils → fixtures, and then consume via data-driven tests. JSON for most cases, CSV for tabular inputs, and Excel only when stakeholders insist—often pre-converted to JSON for CI stability."

Storage layout

```
/testdata
  users.json
  users.csv
  users.xlsx
/utils/data
  readJson.js
  readCsv.js (we have used papaparse module)
  readXlsx.js (we have used SheetJS module)
/fixtures
  data.fixture.js
```

We can consume them whenever we need test data.

24. How do you generate reports from Playwright (Allure, HTML, etc.)?

I use Playwright's multi-reporter setup: built-in HTML for humans, JUnit for CI dashboards, and Allure for rich analytics. For large suites I also use the blob reporter to merge results from shards."

1) Built-in HTML (human-readable)

Config

```
// playwright.config.ts
import { defineConfig } from '@playwright/test';
export default defineConfig({
  reporter: [
    ['list'],
    ['html', { outputFolder: 'playwright-report', open: 'never' }],
  ],
  use: { trace: 'on-first-retry', video: 'retain-on-failure', screenshot: 'only-on-failure' },
});
```

Run & open

```
npx playwright test
npx playwright show-report      # opens playwright-report
```

2) JUnit (CI trends: Jenkins/GitHub/ Azure, etc.)

Add reporter

```
reporter:
[
  ['list'],
  ['html', { outputFolder: 'playwright-report', open: 'never' }],
  ['junit', { outputFile: 'junit-results.xml' }],
],
```

CI then parses junit-results.xml for pass/fail trends.

3) Allure (rich dashboards)

Install

```
npm i -D allure-playwright allure-commandline
```

Config

```
reporter: [  
  ['list'],  
  ['html', { outputFolder: 'playwright-report', open: 'never' }],  
  ['junit', { outputFile: 'junit-results.xml' }],  
  ['allure-playwright'], // writes to ./allure-results  
,  
]
```

Generate & open

```
npx allure generate allure-results --clean -o allure-report  
npx allure open allure-report
```

(Tip: keep trace: 'on-first-retry' so HTML/Allure attach traces/videos on failures.)

4) Optional: JSON for custom analytics

```
reporter: [  
  ['json', { outputFile: 'playwright-results.json' }],  
  ['html'], ['junit'], ['allure-playwright'],  
,  
]
```

One-liner answer

"Use multi-reporters: HTML for humans, JUnit for CI, Allure for insights; enable traces/videos; and for sharded runs, blob merge (HTML/JUnit) or aggregate allure-results for Allure.

25. How do you run tests across different browsers and devices in Playwright?

I run the same suite across multiple browsers/devices using Playwright projects. Each project defines its browser (Chromium/Firefox/WebKit) or a device profile (e.g., iPhone), and Playwright executes them in parallel. I select projects via CLI and keep everything driven from config—no code changes between targets."

1) Configure projects (browsers + devices)

```
// playwright.config.js
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  // Common defaults for all projects
  use: {
    baseURL: process.env.BASE_URL || 'https://app.example.com',
    trace: 'on-first-retry',
    video: 'retain-on-failure',
    screenshot: 'only-on-failure',
    headless: true,
  },

  // Run the same tests across these projects in parallel
  projects: [
    // Desktop browsers
    { name: 'chromium', use: { browserName: 'chromium' } },
    { name: 'firefox', use: { browserName: 'firefox' } },
    { name: 'webkit', use: { browserName: 'webkit' } },

    // Real Chrome/Edge channels (optional)
    { name: 'chrome', use: { channel: 'chrome' } },
    { name: 'msedge', use: { channel: 'msedge' } },

    // Mobile emulation (touch, DPR, UA, viewport, etc.)
    { name: 'Pixel 7', use: { ...devices['Pixel 7'] } },
    { name: 'iPhone 14', use: { ...devices['iPhone 14'] } },

    // Responsive desktop sizes (no device UA)
    { name: 'desktop-1280', use: { viewport: { width: 1280, height: 800 } } },
    { name: 'desktop-1920', use: { viewport: { width: 1920, height: 1080 } } },
  ],
});
```

Notes:

- `browserName` uses Playwright's bundled engines; `channel` runs your installed Chrome/Edge.
- Device profiles from `devices` set UA, viewport, DPR, `isMobile`, `hasTouch`, etc.

2) Run all or target specific projects (CLI)

Run across ALL configured projects (parallel by default)

```
npx playwright test
```

Run only specific browsers

```
npx playwright test --project=chromium --project=firefox
```

Run only mobile projects

```
npx playwright test --project="iPhone 14" --project="Pixel 7"
```

Combine with tags or a single file

```
npx playwright test tests/checkout.spec.js --project=chromium -g "@smoke"
```

3) Per-test overrides (when only one test needs a device/geo)

```
import { test, expect } from '@playwright/test';
```

```
test.use({ viewport: { width: 375, height: 812 }, isMobile: true, hasTouch: true });
```

```
test('mobile-only banner appears', async ({ page }) => {  
  await page.goto('/');  
  await expect(page.getByTestId('mobile-banner')).toBeVisible();  
});
```

Geo/locale/timezone when needed:

```
test.use({  
  geolocation: { latitude: 28.6139, longitude: 77.2090 },  
  permissions: ['geolocation'],  
  locale: 'en-IN',  
  timezoneId: 'Asia/Kolkata',  
});
```

4) Best practices I follow

- Projects, not branches: one codebase → many targets.

- Use stable locators (getByRole, getByLabel, data-testid) so selectors work across engines.
- Parallel + shards in CI for speed (projects × workers).
- Keep artifacts (HTML/JUnit, trace, video) enabled for all projects to debug browser-specific issues.
- If a test is truly browser/device-specific, tag it and run selectively.

Advanced Playwright

26. How do you implement API testing with Playwright?

"Playwright has a first-class API client (APIRequestContext) that I use for pure API tests and to combine API + UI flows. I centralise auth/baseURL in config/fixtures, validate payloads, and keep tests deterministic (mock when needed)."

1) Quick API tests with built-in request fixture

```
// tests/api/users.spec.js
import { test, expect } from '@playwright/test';

test('GET user', async ({ request }) => {
  const res = await request.get('/api/users/2');
  expect(res.ok()).toBeTruthy();
  const body = await res.json();
  expect(body.data.id).toBe(2);
});

test('POST create user', async ({ request }) => {
```

```

const res = await request.post('/api/users', {
  data: { name: 'Alice', job: 'QA' }
});
expect(res.status()).toBe(201);
const body = await res.json();
expect(body.name).toBe('Alice');
});

```

Config (baseUrl, headers, auth):

```

// playwright.config.js
export default {
  use: {
    baseUrl: process.env.BASE_URL || 'https://reqres.in',
    extraHTTPHeaders: {
      Authorization: `Bearer ${process.env.API_TOKEN ?? ''}`
    }
  }
};

```

2) Reusable API client via fixture (clean tests)

```

// fixtures/api.fixture.js
import { test as base, expect } from '@playwright/test';

export const test = base.extend({
  api: async ({ request }, use) => {
    // optional: ping/healthcheck or token refresh here
    await use({
      getUser: (id) => request.get(`/api/users/${id}`),
      createUser: (payload) => request.post('/api/users', { data: payload }),
    });
  },
});
export { expect };

// tests/api/clean.spec.js
import { test, expect } from '../fixtures/api.fixture.js';

test('create + fetch user (API)', async ({ api }) => {
  const create = await api.createUser({ name: 'Bob', job: 'Tester' });

```

```
expect(create.status()).toBe(201);

const id = (await create.json()).id;
const get = await api.getUser(id);
expect(get.ok()).toBeTruthy();
});
```

3) Combine API + UI (fast & realistic)

- Seed via API, then verify in UI (or the reverse).

```
import { test, expect } from '@playwright/test';

test('seed user by API then verify in UI', async ({ request, page }) => {
  const { id } = await (await request.post('/api/users', {
    data: { name: 'Carol', job: 'QA' }
  })).json();

  await page.goto('/users');
  await expect(page.getByText(`Carol`)).toBeVisible();
});
```

One-liner: *“Use Playwright’s APIRequestContext for fast API tests, seed or verify UI when needed, validate contracts, and mock externals to keep runs deterministic.”*

27. What is Playwright Test Runner and how is it different from Jest or Mocha?

What is Playwright Test Runner?

“Playwright Test Runner (@playwright/test) is Playwright’s built-in E2E test framework. It bundles the test DSL (test, expect), web-first assertions (auto-wait), fixtures/DI, parallel workers,

projects (cross-browser/device), retries, and rich artifacts (trace, video, screenshots) — all optimized for browser automation.”

How it differs from Jest or Mocha

1. Purpose-built for browsers vs. general-purpose
 - Playwright Test: designed for E2E/UI/API with real browsers.
 - Jest/Mocha: general test runners (unit/integration); need extra libs for browser automation.
2. Web-first auto-waiting & assertions
 - Playwright: `expect(locator).toBeVisible()` auto-waits for visible/attached/idle states.
 - Jest/Mocha: assertions are generic; no built-in DOM auto-wait (you add waits or helpers).
3. Built-in browser lifecycle & isolation
 - Playwright: manages Browser → Context (per test) → Page isolation out of the box.
 - Jest/Mocha: you wire Puppeteer/Playwright yourself; isolation is manual.
4. First-class fixtures (Dependency Injection)
 - Playwright: fixtures (page, context, custom fixtures) with test/worker scope.
 - Mocha/Jest: rely on hooks; DI/fixtures aren't a native concept (you build your own).

5. Projects for cross-browser/device matrix

- Playwright: run same suite across Chromium/Firefox/WebKit and mobile profiles in one command.
- Jest/Mocha: no native cross-browser matrix; requires custom orchestration.

6. Parallelism, retries, sharding built in

- Playwright: workers, retries, --shard, expect timeouts — all native.
- Jest/Mocha: parallelism varies (Jest has workers for tests; Mocha needs flags/tools); sharding/ retries typically external.

7. Artifacts & debugging

- Playwright: trace viewer, video, screenshots, PWDEBUG inspector out of the box.
- Jest/Mocha: reporters exist, but rich browser timeline/trace isn't native.

8. Reporting

- Playwright: HTML, JUnit, JSON, blob merge for sharded CI.
- Jest/Mocha: reporters exist, but merging E2E artifacts often bespoke.

Tiny example (Playwright Test)

```
import { test, expect } from '@playwright/test';
```

```
test('user can log in', async ({ page }) => {  
  await page.goto('/login');  
  await page.getByLabel('Email').fill('user@example.com');  
  await page.getByLabel('Password').fill('P@ssw0rd!');  
});
```

```
await page.getByRole('button', { name: 'Sign in' }).click();
await expect(page).toHaveURL(/dashboard/); // web-first auto-wait
});
```

One-liner takeaway

“Playwright Test is an all-in-one E2E runner with web-first waits, fixtures, cross-browser projects, artifacts, and CI primitives — whereas Jest/Mocha are general runners that require additional tooling to achieve the same E2E capabilities.”

28. How do you create custom commands or utilities in Playwright?

“Playwright doesn’t have a Cypress-style Cypress.Commands.add, but we can achieve the same using utility functions or extending fixtures/page objects. The goal is to extract reusable actions so tests stay clean and DRY.”

1) Utility functions (simplest)

Put common actions in a helper file.

```
// utils/customActions.js
export async function login(page, email, password) {
  await page.goto('/login');
  await page.getByLabel('Email').fill(email);
  await page.getByLabel('Password').fill(password);
  await page.getByRole('button', { name: 'Sign in' }).click();
}
```

Use in tests:

```
import { test, expect } from '@playwright/test';
```

```
import { login } from '../utils/customActions.js';

test('login via utility', async ({ page }) => {
  await login(page, 'admin@example.com', 'Admin@123');
  await expect(page).toHaveURL(/dashboard/);
});
```

2) Extend Page Objects (OOP style)

Wrap actions inside your page model.

```
// pages/LoginPage.js
export class LoginPage {
  constructor(page) {
    this.page = page;
    this.email = page.getByLabel('Email');
    this.password = page.getByLabel('Password');
    this.submit = page.getByRole('button', { name: 'Sign in' });
  }
  async loginAs(username, pass) {
    await this.email.fill(username);
    await this.password.fill(pass);
    await this.submit.click();
  }
}
```

Usage:

```
import { test } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage.js';

test('login via POM command', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.loginAs('admin@example.com', 'Admin@123');
});
```

3) Extend Fixtures (DI style, very Playwright-native)

Create custom fixtures that provide ready-to-use helpers.

```
// fixtures/custom.fixture.js
import { test as base } from '@playwright/test';

export const test = base.extend({
  loginAsAdmin: async ({ page }, use) => {
    await page.goto('/login');
    await page.getByLabel('Email').fill('admin@example.com');
    await page.getByLabel('Password').fill('Admin@123');
    await page.getByRole('button', { name: 'Sign in' }).click();
    await use(page);
  },
});
export { expect } from '@playwright/test';
```

Use in test:

```
import { test, expect } from '../fixtures/custom.fixture.js';

test('dashboard visible after admin login', async ({ loginAsAdmin }) => {
  await expect(loginAsAdmin.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
});
```

Best practices

- Keep atomic helpers in utils/ (e.g., login, seed data, API calls).
- Use Page Objects for modeling screens.
- Use Fixtures for login sessions or heavy setup (clean isolation, worker vs test scope).

29. How do you use test hooks (`beforeAll`, `beforeEach`, `afterAll`, `afterEach`)?

"Playwright Test provides standard hooks — beforeAll, beforeEach, afterAll, afterEach. I use them to manage setup/teardown logic at the suite or test level. The key is to keep them fast, isolated, and idempotent so they don't introduce flakiness."

1) beforeAll / afterAll → suite-level

- Runs once per test.describe block (or once in the file if top-level).
- Good for expensive setup (e.g., DB seed, global auth state).
- Avoid UI actions here unless all tests can reuse the result.

```
import { test, expect } from '@playwright/test';
```

```
let context, page;
```

```
test.beforeAll(async ({ browser }) => {  
  context = await browser.newContext();  
  page = await context.newPage();  
  await page.goto('/login');  
});
```

```
test.afterAll(async () => {  
  await context.close();  
});
```

2) beforeEach / afterEach → per test

- Runs before/after every test.
- Good for isolated state setup, like navigating to a start page, resetting cookies, or cleaning test data.

```
test.beforeEach(async ({ page }) => {  
  await page.goto('/');  
});
```

```
test.afterEach(async ({ page }) => {  
  // e.g., clear localStorage if needed  
  await page.evaluate(() => localStorage.clear());  
});
```

3) Combined example with describe

```
test.describe('Shopping Cart', () => {  
  test.beforeAll(async ({ request }) => {  
    // Seed test data once (API is faster than UI)  
    await request.post('/api/seed-cart', { data: { user: 'qa' } });  
  });  
  
  test.beforeEach(async ({ page }) => {  
    await page.goto('/cart');  
  });  
  
  test('add item to cart', async ({ page }) => {  
    await page.getByText('Add to Cart').click();  
    await expect(page.getByTestId('cart-count')).toHaveText('1');  
  });  
  
  test.afterEach(async ({ page }) => {  
    await page.reload(); // reset between tests  
  });  
  
  test.afterAll(async ({ request }) => {  
    await request.post('/api/cleanup-cart', { data: { user: 'qa' } });  
  });  
});
```

Best practices I follow

- Keep tests independent — don't rely on state from beforeAll unless it's immutable.
- Prefer fixtures for reusable logins/auth instead of hooks.
- Use API calls in hooks (faster + more reliable) rather than UI for setup/teardown.

- Always clean up in `afterAll/afterEach` to avoid polluting other tests.

One-liner:

“I use `beforeAll/afterAll` for one-time suite setup/cleanup, `beforeEach/afterEach` for per-test isolation, and prefer API + fixtures to keep tests deterministic and fast.”

30. How do you handle Shadow DOM and iframe elements?

Shadow DOM

“Playwright natively pierces open Shadow DOM, so most locators ‘just work’. I use semantic locators first; if needed, I scope down with normal CSS. Closed shadow roots aren’t accessible (by design).”

Basics (open shadow roots work out of the box):

```
// Semantic locator (recommended)
await page.getByRole('button', { name: 'Add to cart' }).click();
```

```
// CSS inside shadow roots also works
await page.locator('product-card >> .buy button').click();
```

Scoped, readable selectors:

```
const card = page.locator('product-card'); // host
await card.getByText('Premium Plan').click(); // inside its shadow DOM
```

Notes

- No special flags needed for open shadow roots.
- Closed shadow DOM cannot be traversed by any DOM API; coordinate with devs to avoid closed shadows in test-critical areas or expose test hooks (events, `data-testid`, public methods).

Reliability tips (both Shadow DOM & iframes)

- Prefer `getByRole` / `getByLabel` / `getByTestId` for resilient selectors.

Use assertions that auto-wait:

```
await expect(page.frameLocator('#f').getByText('Loaded')).toBeVisible();
```

For elements that render late, wait on frame navigation/content:

```
await page.frameLocator('#f').locator('form').waitFor();
```

- Keep actions scoped (store a parent locator, then search inside).

“Open Shadow DOM works out of the box with Playwright locators; closed does not. For iframes, use `frameLocator` (chainable, auto-waiting) to target elements reliably—even in nested or cross-origin frames.”

Scenario / Problem-Solving

31. You need to test an application that triggers background API calls—how would you validate responses?

“When UI actions trigger background API calls (XHR/fetch/GraphQL/WebSocket), I validate the exact network response and correlate it with visible UI state. I prefer event-based waits over sleeps, and I stub flaky externals when needed.”

1) Wait for the exact network call (XHR / fetch)

```
import { test, expect } from '@playwright/test';

test('UI triggers background fetch and renders result', async ({ page }) => {
  await page.goto('/reports');

  // Click → triggers background /api/report call (no navigation)
  const [res] = await Promise.all([
    page.waitForResponse(r =>
      r.url().endsWith('/api/report') && r.request().method() === 'GET'
    ),
    page.getByRole('button', { name: 'Load Report' }).click(),
  ]);

  expect(res.ok()).toBeTruthy();
  const body = await res.json();
  expect(body.total).toBeGreaterThan(0);

  // Correlate with UI
  await expect(page.getByTestId('report-total')).toHaveText(String(body.total));
});
```

Why: event-based synchronization; validates response and UI outcome.

One-liner

"I synchronize on the network event that matters (waitForResponse), assert payload and body, handle eventual consistency with expect.poll, and—when needed—stub the endpoint for deterministic runs, always tying the API result back to visible UI state."

32. If Playwright doesn't support a browser feature directly, how do you extend functionality?

Interview-style Answer

"If Playwright doesn't support a browser feature directly, I extend functionality using the browser's native APIs and Playwright's low-level primitives. Depending on the case, I'll use:

`page.evaluate()` / `frame.evaluate()` – run raw JavaScript in the browser context.

```
const battery = await page.evaluate(() => navigator.getBattery());  
console.log(battery.level);
```

“If Playwright lacks direct support, I extend via evaluate (DOM APIs), CDP sessions (low-level Chrome features), or Node integrations, wrapping them in fixtures/utilities so my test suite stays clean and maintainable.”

32. How do you combine Playwright with AI/GenAI tools (like Copilot or MCP server) for automation efficiency?

Using Copilot to generate test cases, page objects, and locators quickly.

Leveraging AI to suggest stable selectors, triage flaky tests, and summarize test failures.

Integrating MCP tools with Playwright to seed data, toggle feature flags, or run targeted tests via natural language prompts.

In short: AI reduces authoring and debugging time, while MCP lets you orchestrate test workflows more efficiently.